

# M.O.A.B

## Grid Searcher

Complete build plan and specification

---

A dedicated R Shiny application for methodically grid-searching football prediction rules across any market, tuning them safely, validating them honestly, and pushing winners to production.

PREPARED FOR

**Chukwudi Samuel Ogbuta**

COMPILED

**01 August 2026**

# Contents

|           |                                          |
|-----------|------------------------------------------|
| <b>1</b>  | Purpose and background                   |
| <b>2</b>  | Why build this now                       |
| <b>3</b>  | System architecture at a glance          |
| <b>4</b>  | Data loading and file management         |
| <b>5</b>  | Tuning, validation, and test splits      |
| <b>6</b>  | Stage 1 — Grid search engine             |
| <b>7</b>  | Stage 2 — Intuition library              |
| <b>8</b>  | Feature engineering catalog              |
| <b>9</b>  | Market tiers and stats-coverage handling |
| <b>10</b> | Rule-family generalization               |
| <b>11</b> | The is_main_league filter                |
| <b>12</b> | Window sweeps                            |
| <b>13</b> | Drift detection and monitoring           |
| <b>14</b> | Pushing to production (moab_analysis.R)  |
| <b>15</b> | Build timeline and phasing               |
| <b>16</b> | Improvement backlog (deferred items)     |
| <b>17</b> | Definitions and glossary                 |

# 1. Purpose and background

MOAB (Mother of All Boards) is a production football intelligence platform that scrapes fixtures, form, head-to-head, and per-match statistics from Flashscore, enriches them with computed features, and issues betting-market predictions through rule-based analysis modules. The current production rule for Over 2.5 FT was found manually — thresholds tuned by inspection, filters added iteratively, results eyeballed. That worked to prove the concept but does not scale.

The Grid Searcher is a dedicated Shiny application that formalizes the entire rule-discovery process. It loads sim-ready RDS files, sweeps every combination of thresholds against historical results, applies an intuition library of hand-crafted filters, validates winners on hold-out data, monitors deployed rules for drift, and produces production-ready R code for insertion into `moab_analysis.R`. It exists so Chidi can spend hours on his M2 studies while grid searches finish in minutes, not manual weeks.

The 2026-27 season starts **14 August 2026** (La Liga first, Bundesliga last on 28 August). Chidi's calibration plan targets three Sundays (6, 13, 20 September) for initial data collection, with meaningful rule commitment targeted by round 4, confident deployment by round 8-10, and staking-appropriate confidence from round 10 onwards. The Grid Searcher must be operational — at least Stage 1 — before R4.

# 2. Why build this now

Three reasons the build should happen in August, before season data arrives:

- **The design is fresh.** The full specification was worked out in conversation and must be captured while every decision is remembered.
- **Development on stale data is safe.** An existing pre-parity sim-ready file (structure identical, feature set narrower) is enough to build the app skeleton, wire up loading, splits, and Stage 1 mechanics.
- **Season pressure is real.** Once R1 lands on 6 September, building under time pressure while trying to also calibrate rules is a recipe for bugs and half-decisions. Better to have the app ready and just feed it data as it arrives.

## TIMING CONSTRAINT

Skeleton work uses the old sample RDS. Anything requiring new-format fields (goal times, 1H/2H stat splits, expanded WANTED\_STATS) waits until R1 data is in hand in early September. Full development is completed round-by-round through R4.

### 3. System architecture at a glance

The Grid Searcher is a standalone Shiny app sitting alongside the existing MOAB stack. It reads sim-ready RDS files produced by the simulator's scrape flow (which enriches predictions with results), and it optionally writes back to `moab_analysis.R` when a rule is approved for production.

#### Data flow

Simulator scrapes results and produces `MoabSim_Ready_*.rds` files. Each file is a list of enriched fixtures, one per prediction. Each fixture carries its own `ef$fixture` data frame (teams, league, and `result_*` fields after scrape), `ef$home_form` (list of 7 recent matches for the home team, each with `is_main_league`, goals, HT/FT, stats, stats\_1h, stats\_2h, goal\_times), `ef$away_form` (same for away), and `ef$h2h` (list of 5 head-to-heads with identical structure).

The Grid Searcher loads N such files, tags each with a role (tune / validate / test / exclude), extracts features per fixture, runs grid searches, applies intuition-library filters, validates on hold-out sets, and produces ranked leaderboards.

#### Component overview

|                        |                                                                                                                                |
|------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| <b>File loader</b>     | Multi-RDS drag-drop with role tagging and session save/load.                                                                   |
| <b>Feature builder</b> | Per-fixture derivation of quantitative features from form and H2H.                                                             |
| <b>Stage 1 engine</b>  | Threshold grid sweep on a chosen rule shape; optional window and rule-family axes.                                             |
| <b>Stage 2 library</b> | Persistent intuition rules with category tags; applied on top of Stage 1 winners.                                              |
| <b>Validator</b>       | Re-grades shortlist on held-out rounds; flags overfitting via fit-vs-test gap.                                                 |
| <b>Drift monitor</b>   | Tracks deployed rules against rolling-round performance; alerts when tolerance exceeded.                                       |
| <b>Code emitter</b>    | Generates <code>get_marketname()</code> function matching <code>moab_analysis.R</code> pattern with tuned thresholds baked in. |
| <b>Diff + push</b>     | Shows what would change in <code>moab_analysis.R</code> , user confirms, timestamped backup before write.                      |

## 4. Data loading and file management

The app supports loading multiple sim-ready RDS files simultaneously. Each loaded file appears in a management table showing filename, auto-parsed round label, fixture count, date range, and current role.

### Role tagging

Each file gets an explicit role via a per-row dropdown:

- **Tune** — file contributes to the grid search's fit set.
- **Validate** — file used to shortlist Stage 1 candidates before Stage 2 exploration (middle checkpoint).
- **Test** — file held completely out of tuning; used to verify final rule performance.
- **Exclude** — file loaded but temporarily disabled for this run.

Explicit dropdown was chosen over auto-split by date order because transparency matters more than convenience — the user always knows which files fed which stage.

### Sessions

The current file list and role assignments can be saved as a named session (e.g. "R1-R4 calibration"). Sessions persist to disk and can be reloaded to reproduce an exact grid search configuration. This enables replay analysis — "how would today's rules have performed if I'd tuned them at R4?" versus "at R7?"

### Adding data mid-season

New sim-ready files are attached to the existing list; the user re-tags roles as needed. The typical weekly flow is: last week's Test file becomes this week's Tune file; the newest week becomes the new Test. Existing files stay loaded; no re-uploading required.

## 5. Tuning, validation, and test splits

Grid search is fundamentally a fitting process. Without discipline, it will find rules that fit random noise in the training data and fail immediately on new fixtures. The three-way split is the primary defence.

### The four-round standard

The minimum viable configuration uses **four rounds of data**:

- **Rounds 1 to N-2** (Tune): fed into Stage 1 grid search. Rules are chosen based on their performance here.
- **Round N-1** (Validate): Stage 1 shortlist is re-graded on this round. Rules that collapse are dropped. Stage 2 exploration runs against this shortlist.
- **Round N** (Test): the final approved rule is graded here for the first time. This is the honest expectation of live performance.

As the season progresses, older rounds join the Tune set and the newest round becomes Test. By round 10, the Tune set is ~7 rounds, Validate is 1, Test is 1 — providing enough sample for confident rule commitment.

## Why four rounds not three

A three-round split (2 tune, 1 test) leaves no room to catch overfitting between Stage 1 and Stage 2. The validate round is where Stage 1 winners are pruned before Stage 2 filters are layered on. Without it, Stage 2 fits on the same data Stage 1 already fit, compounding the overfitting risk.

### SAMPLE FLOOR

Sample-size floors for Stage 1 winners must be configurable per market. The default target is 100+ picks after Stage 1 (before Stage 2 tightening), with the Stage 2 output reducing to acceptable flex volume (dozens of picks with  $\leq 4$  losses).

## 6. Stage 1 — Grid search engine

Stage 1 is the quantitative sweep. For a chosen market, the engine iterates over every combination of thresholds for the rule's features and records the resulting pick volume, win count, loss count, and hit rate for each combination.

### Per-market rule specification

Before any grid search runs, the user is shown the full feature list for the chosen market — separated into **goals-only** and **stats-augmented** variants — and must confirm it. The list includes home form features, away form features, and H2H features. Nothing sneaks in or gets omitted without explicit sign-off.

### Search axes

The grid search may sweep across multiple orthogonal dimensions in a single run:

- **Threshold values** for each feature (e.g. `h_scored_n5` from 2 to 5).
- **Window sizes** per feature (typically 3, 5, or 7 matches).
- **Rule families** —  $\geq$  threshold, range, difference, ratio, combined-sum (see Section 10).

A full mixed search across all axes yields large combination counts (millions of rules) but is computationally cheap in vectorized R — a few minutes per market on a laptop. The compute budget is not the constraint; the interpretation budget is, so the app must automatically deduplicate equivalent rules (same picks/wins/losses across all rounds) before presenting the leaderboard.

### Output — dual leaderboard

Stage 1 produces two ranked leaderboards side-by-side:

- **Picked-fixtures leaderboard** (primary) — for each rule: picks, wins, losses, hit rate. Sortable by hit rate, volume, or baseline-relative lift.
- **Rejected-fixtures leaderboard** (dark counterpart) — for each rule: fixtures rejected, and what proportion of rejected fixtures would have won anyway. Reveals whether a rule is genuinely selective or overly conservative.

The dual view enables two evaluation questions instead of one: "is this rule accurate?" (picked leaderboard) AND "is this rule leaving winners on the table?" (rejected leaderboard). A rule that picks 40 fixtures at 90% but rejects 200 that would have won at 75% is a filtering failure — the rescue mechanisms in Stage 2 and identical-team analysis exist specifically to recover those false negatives.

The user selects a Stage 1 winner (or a small set) to carry forward into Stage 2 exploration and rescue analysis.

## 7. Stage 2 — Intuition library

Stage 2 layers additional filters on top of a Stage 1 winner. Its purpose is to squeeze the loss count down to the  $\leq 4$  flex margin while preserving useful volume. Where Stage 1 is a mechanical sweep, Stage 2 is a curated library of intuition-based rules the user has thought of.

### Library structure

Each rule in the library carries:

- **Rule name** — a short human label.
- **Category** — one of: momentum, consistency, standings, draw-related, H2H, discipline, home/away split (list is extensible).
- **Rule logic** — the actual filter code, using the same feature helpers as Stage 1.
- **Tunable parameters** — thresholds within the rule that can be grid-searched during Stage 2 exploration.
- **Status per market** — active, rejected-for-market-X, untested-for-market-Y.

### Application flow

For a selected Stage 1 winner, Stage 2 runs a second grid search restricted to the fixtures the Stage 1 rule picked. It sweeps single filters from the library and also combinations of 2-3 filters (matching the current Over 2.5 pattern which combines two second-stage filters). The user can restrict Stage 2 to specific categories, enabling comparisons like "only momentum filters help this market" vs "draw filters add nothing."

### Rejected rules stay

A filter that fails for one market is not deleted — it is marked rejected-for-that-market. The next market that runs Stage 2 automatically tests it again. A rank-gap filter that was rejected for Over 2.5 might be gold for the Cards Over 3.5 market. Nothing gets forgotten; the library only grows.

#### DISCIPLINE

Stage 2 is fit on the Tune set exactly like Stage 1, then re-graded on Validate. Final combined-rule performance is measured on Test. The library will grow, and with more filters comes more overfitting risk — the hold-out discipline is what stops the growing library from silently becoming a lottery machine.

## 8. Feature engineering catalog

Flashscore has been exhaustively scraped — there is nothing more to add on the raw-data side. Signal now lives in how the existing fields are combined into derived features. This section catalogs the features to make available to Stage 1 and Stage 2 grid searches, organized by source.

### Goals-based features (from form and H2H FT/HT)

- **Count-based:** `h_scored_n5`, `h_conceded_n5`, `h_btts_n5`, `h_over25_n5`, `h_clean_sheets_n5`, `h_matches_with_2plus_n5`
- **Mean-based:** `h_avg_goals_scored_n5`, `h_avg_goals_conceded_n5`, `h_avg_ht_goals_n5`
- **Volatility:** variance of goals scored (`h_score_variance_n5`), high-scoring game frequency
- **Weighted quality:** `h_avg_goals_when_scored_n5` (mean goals per scoring game, excludes goalless matches)
- **Momentum shifts:** last-3 vs last-7 form comparison — swing detection

### Stats-based features (from scraped match stats)

- **Attacking output:** `h_avg_xg_n5`, `h_avg_shots_n5`, `h_avg_sot_n5`, `h_avg_big_chances_n5`, `h_avg_touches_opp_box_n5`
- **Defensive:** `h_avg_shots_conceded_n5`, `h_avg_xga_n5`
- **Tempo:** `h_avg_possession_n5`, `h_avg_total_shots_n5`
- **Discipline:** `h_avg_fouls_n5`, `h_avg_cards_n5`
- **Set-piece:** `h_avg_corners_n5`, `h_avg_free_kicks_n5`

### Head-to-head features

- **H2H BTTS rate:** proportion of last 5 meetings where BTTS hit
- **H2H goal expectancy:** mean total goals in last 5 meetings
- **H2H blank rate:** proportion of last 5 meetings where at least one team failed to score (catches teams that blank each other)
- **H2H clean-sheet distribution:** home vs away clean-sheet frequency

### Meeting-point features (interactions)

- **Attack × defense:** home's scoring rate multiplied by away's conceding rate — a raw mismatch signal
- **Standings gap:** `home_rank` minus `away_rank` — bigger gap suggests mismatched game
- **Form differential:** home last-5 points minus away last-5 points
- **League-normalized attack:** `h_avg_goals_n5` divided by league average — cross-league comparability

### Poisson-derived features (existing in `moab_analysis.R`)

MOAB already computes Poisson-based goal expectancies as part of the current Over 2.5 gate. These are kept as first-class features in the grid searcher, not gates — the grid decides how much weight they carry alongside counting and mean features.

- **lambda\_h, lambda\_a:** recency-weighted expected goals per team, computed as  $(\text{team\_scoring\_rate} \times \text{opponent\_conceding\_rate} \times \text{league\_avg})$
- **total\_lambda:** sum of home and away lambdas — expected goals for the match
- **p\_over\_25, p\_over\_15, p\_btts:** probability of each market hitting based on Poisson distribution of the two lambdas across score grids
- **p\_home\_win, p\_draw, p\_away\_win:** match outcome probabilities from the same score grid

## Goal-timing features (from scraped goal\_times)

- **Late-goal tendency:** proportion of goals scored after minute 75
- **Early-goal tendency:** proportion of goals in first 20 minutes
- **Second-half scoring rate:** goals scored between minute 46 and 90
- **Comeback frequency:** matches won or drawn after conceding first (derivable from HT/FT plus goal times)

## Opponent-strength features (from cross-referencing form vs standings)

Standard grid search treats all past opponents equally — a team that scored 4/5 against weak defenses looks identical to one that scored 4/5 against strong defenses. Opponent-strength features restore this context by cross-referencing each historical opponent's current standings position, ppg, and goal difference to characterize opposition quality. Limited to last 3-4 main-league opponents to keep the standings-as-proxy reasonably accurate (opponent rank shift within 4 weeks is typically small).

- **Average opponent strength:**  $h\_avg\_opp\_rank\_n4$ ,  $h\_avg\_opp\_ppg\_n4$ ,  $h\_avg\_opp\_gd\_n4$
- **Sub-dimensional opponent quality:**  $h\_avg\_opp\_attack\_n4$  (mean of opponents' goals-scored rates),  $h\_avg\_opp\_defense\_n4$  (mean of opponents' goals-conceded rates),  $h\_avg\_opp\_tempo\_n4$  (mean of opponents' shots/game)
- **Wins vs strong opposition:**  $h\_wins\_vs\_top6\_n4$ ,  $h\_conceded\_vs\_bottom6\_n4$  (high value = red flag)
- **Strength-weighted goals:**  $h\_scored\_weighted\_by\_opp\_defense\_n4$  (scoring against weak defenses discounted, against strong defenses amplified) — catches paper tigers and hidden gems

## Identical-team analysis features

The upgrade beyond average opponent strength. For each fixture, characterize the upcoming opponent along multiple dimensions (attack, defense, tempo), then find which of the team's recent 3-4 opponents most closely resembles it in profile. Pay special attention to how the team performed in that specific similar match — matchup dynamics matter more than raw team quality.

### Level 1 — profile summary (broader signal, more stable)

- **Performance vs similar opposition:** `h_avg_ft_for_vs_similar_defenses`, `h_avg_ft_against_vs_similar_attacks`
- **Win rate vs profile-similar:** `h_win_rate_vs_similar_opposition`
- **Market-specific outcome rate:** `h_btts_rate_vs_similar`, `h_over25_rate_vs_similar`

### Level 2 — pattern match (sharper but noisier, uses one match)

- **Most similar match:** `h_most_similar_match_ft_total`, `h_most_similar_match_btts`, `h_most_similar_match_result`
- **Similar-match count:** `h_similar_matches_count_n4` (how many of last 4 opponents were similar to upcoming — higher count = stronger pattern-match confidence)

### How similarity is computed

Each opponent gets a feature vector — (attack\_strength, defense\_strength, tempo, discipline) — derived from league-normalized recent form. The upcoming opponent gets the same vector. "Similarity" is just feature-space distance (Euclidean). Historical matches can then be weighted by similarity in aggregate features, or the single most-similar match can be surfaced for pattern-match features. Both team profiles are computed independently, so a fixture has TWO parallel signals (home's read on facing away-type teams; away's read on facing home-type teams). Grid search can require both to agree before firing.

### Rescue mechanism — the standout use case

Identical-team analysis is not only a filter (Stage 2 tightening). Its most valuable role is as a **rescue mechanism** for fixtures the standard rules would reject. Standard rules are conservative — they reject anything that doesn't hit all thresholds and leave winners on the table. The grid searcher must include a "rejected-fixtures leaderboard" showing: for each standard rule, what proportion of rejected fixtures would have won anyway. Then evaluate whether identical-team features can discriminate winners inside the rejected pile. If rescued fixtures win at 70%+, identical analysis becomes a standalone rescue rule. If at 50-60%, it is noise. This turns false negatives into recoverable volume without compromising the  $\leq 4$  losses discipline.

### Timing constraint

Requires at least 4 main-league matches in each team's form window. Not computable in rounds 1-4 (form window still dominated by friendlies and previous-season tail). Feature engineering and grid-searching for identical-team analysis begins from **round 5 onwards**, with more robust signal by round 8+. This is a separate calibration phase running in parallel to standard rule refinement.

#### ORTHOGONALITY

Feature orthogonality matters. `h_btts_n5` is heavily correlated with `h_scored_n5` + `h_conceded_n5` — including both wastes search compute and inflates apparent rule strength. The app should warn when correlated pairs are enabled together, and offer an orthogonal-only mode.

## 9. Market tiers and stats-coverage handling

Markets differ in what data they need to be graded and what features can be used in rules. The app groups markets into three tiers with different data handling:

### Tier 1 — Goals-only markets

BTTS, Over 2.5 FT, Over 1.5 FT, HT Draw, Home Team To Score, Away Team To Score, 2H Over 1.5, 2H BTTS, Home Clean Sheet, Away Clean Sheet, Home Win to Nil, Away Win to Nil, Both Halves Under 1.5. Graded from FT and HT scores on the summary page. Every FINAL fixture is gradable. Uses the full sample.

### Tier 2 — Mixed markets

Markets that grade from goals but where stats-augmented rules may add signal. Rules in this tier come in two modes: goals-only (full sample) and stats-augmented (smaller sample where stats are populated). The app runs both and shows a side-by-side comparison so the user can decide whether the smaller sample was worth the added signal.

### Tier 3 — Stats-only markets

Corners Over 9.5, Home Corners Over 4.5, Away Corners Over 3.5, Match Cards Over 3.5, Home Bookings Over 1.5, Away Bookings Over 1.5, Total Shots Over 21.5, SOT Over 7.5. Cannot be graded without stats. Sample automatically shrinks to fixtures where the relevant result\_\* field is non-NA.

## Coverage thresholds

For every stats-based feature the app tracks its coverage across the loaded pool:

- **≥ 80%:** feature freely available for rules; rules that use it drop only the uncovered fixtures during grading.
- **50-80%:** feature available with a warning displayed on the leaderboard row — "this rule drops ~30% of sample."
- **< 50%:** feature disabled for the market to prevent noise.

Sample size per rule (not per grid) is always visible on the leaderboard so rules using different features can be compared fairly.

## 10. Rule-family generalization

The default rule shape used in initial demonstrations was "feature  $\geq$  threshold." Real signal often lives in other rule shapes:

### Range rules

feature BETWEEN low AND high. Excludes both under-performers and mean-reversion candidates. Example: teams scoring in 3-4 of last 5 (not 5/5, which is often followed by a regression). The grid sweeps both low and high bounds.

### Difference rules

feature\_home - feature\_away  $\geq$  threshold. Captures mismatch signal without setting absolute thresholds on either side. Home scores meaningfully more than away, regardless of the specific numbers.

### Ratio rules

feature / league\_average  $\geq$  threshold. Cross-league normalization — a team scoring 1.5 goals per game in a 2.0-league is average, but in a 1.0-league they are exceptional. Ratio rules preserve this distinction.

### Combined-sum rules

feature\_home + feature\_away  $\geq$  threshold. Aggregate signal without requiring extremes from either side individually. Useful for "both teams collectively attacking" style rules.

### Quality-adjusted count rules

Adds a quality gate to a count-based feature. Instead of "scored in 3 of last 5," the rule becomes "scored 2+ goals in 3 of last 5." Distinguishes narrow-margin scorers from genuine attacking teams.

#### CONFIGURABILITY

The user must be able to pick which rule families to include in a grid search. Full mixed search across all families multiplies compute but is still tractable; the dedup pass on the leaderboard remains the key to keeping the output interpretable.

## 11. The is\_main\_league filter

Every form and H2H match carries an is\_main\_league boolean set by the main MOAB scraper. It is TRUE when the match's competition string matches the upcoming fixture's league name — flagging matches from the same competition as the prediction target. Preseason friendlies, cup matches, and continental competitions all receive FALSE.

### When to apply the filter

- **Rounds 1-4:** filter is **disabled**. Preseason friendlies dominate the form window; a strict filter would leave most fixtures with insufficient data. Friendlies at least reflect the current squad.
- **Rounds 5-8:** filter is **optional**. Both filtered and unfiltered leaderboards are shown side-by-side; the user compares.

- **Rounds 9+:** filter **active** for stats-based markets by default. Goals-based markets may stay lenient — goal expectancy carries reasonably across match types.

## Handling variable window size

When the filter is active, some teams may have fewer than the requested window's worth of main-league matches (e.g. team played 3 friendlies within its last 7 matches, leaving only 4 main-league games in the same period).

The app uses **Option C — minimum-window requirement with adaptive fill**:

- Per market, set both a window size (e.g. 5) and a minimum required (e.g. 3).
- Walk back through history (up to a max depth like 10) collecting main-league matches until the window is filled.
- If fewer than the minimum are found, exclude the fixture entirely from picks and surface it in an "insufficient data" tab.
- Feature calculations use whatever count was actually collected — features become rates (proportions) or handle variable n gracefully.

## 12. Window sweeps

Window size — how many recent matches feed a feature — is itself a tunable parameter. Different markets and features respond to different windows:

- **Window 3:** captures immediate momentum. Reactive but noisy. Suits markets sensitive to short-term form shifts (streaks, injuries).
- **Window 5:** balances recency and stability. The default for most markets.
- **Window 7:** captures established patterns. Slower to shift but more reliable for counts (BTTS occurrence, over 2.5 occurrence).

The app sweeps window sizes as an additional grid axis. For each feature in a rule, the grid can either lock all features to a shared window (simpler, more interpretable) or sweep them independently (mixed windows within a single rule).

### Full mixed search vs shared window

The default is **full mixed search** — every feature can use a different window in the same rule. Total combinations grow (roughly  $3^n$  for  $n$  features) but stay tractable in vectorized R. If the winning rule uses mixed windows (e.g. `h_scored` on  $n=7$  with `h_conceded` on  $n=3$ ), it is likely a rule no human would have thought to write — the grid discovers it.

A "shared window only" toggle is available for users who prefer simpler rules or want faster runs on tight compute budgets.

## 13. Drift detection and monitoring

Deployed rules do not stay reliable forever. Team compositions change, tactical trends shift, refereeing patterns evolve. The Grid Searcher includes a monitoring panel that tracks every active rule's rolling performance against its tuned baseline.

### What is tracked per rule

- **Tuned baseline:** hit rate and volume from the original grid search (fit set numbers).
- **Rolling recent performance:** hit rate and volume across the last  $N$  rounds since deployment ( $N$  configurable, default 3).
- **Drift delta:** the gap between current and tuned performance.
- **Per-round history:** full picks/wins/losses log so trends can be visualized.

### Alert conditions

A rule enters an alert state when either:

- Hit rate drops by more than  $X$  percentage points below tuned baseline ( $X$  configurable per rule).
- Rolling volume drops by more than  $Y$  percent from tuned expectation ( $Y$  configurable).
- Loss count in the most recent round exceeds a hard ceiling.

Alerted rules are flagged on a dashboard with a "retune recommended" indicator. The user can then re-run grid search on updated data to either confirm the rule still holds with adjusted thresholds or retire it if the signal has genuinely faded.

## 14. Pushing to production (moab\_analysis.R)

The Grid Searcher can generate production-ready R code from an approved rule and, with user confirmation, write it into `moab_analysis.R`. The push flow is intentionally interactive to prevent silent deployment of broken logic.

### Code emission

After a rule is approved, the app generates:

- A `get_marketname()` function matching the existing pattern in `moab_analysis.R`
- The tuned thresholds baked in as parameters of `DEFAULT_MOAB_THRESHOLDS`
- A comment header documenting: fit-set performance, validation-set performance, test-set performance, and date of tuning
- Registration into the `ALL_MARKETS` list at the bottom of the file

### Diff preview and apply

Before any write happens, the app shows a diff preview: old file contents versus proposed new contents, with additions and removals highlighted. The user visually confirms the diff, then clicks Apply.

On Apply:

- The app creates a timestamped backup: `moab_analysis_backup_YYYYMMDD_HHMM.R`
- The new content is written to `moab_analysis.R`
- A log entry is added to the deployment history within the app

#### SAFETY

The manual paste option (copy code, paste into RStudio) is always available as a fallback. The diff-and-apply flow is a speed optimization; users who prefer manual review can bypass it entirely.

## 15. Build timeline and phasing

The build is phased to align with the 2026-27 season calendar. Each phase produces working functionality; nothing is left half-built between phases.

### Phase 1 — August 2026 (pre-season)

- File loader with role tagging
- Stage 1 grid search engine on one market (BTTS as the reference)
- Basic leaderboard with sort and dedup
- Development uses the existing pre-parity sample RDS

### Phase 2 — September 2026 (R1 arrives)

- Feature engineering module updated for new-parity fields (goal\_times, 1H/2H stats)
- Multi-market support — Over 2.5 and BTTS both operational
- Train/validate/test split flow
- Stage 2 intuition library skeleton with a starter set of filters carried over from moab\_analysis.R

### **Phase 3 — October 2026 (R4 through R6)**

- Full Stage 2 exploration UI
- Window sweep and rule-family generalization
- Coverage-threshold-aware stats-augmented mode
- is\_main\_league filter (activated at round 9)

### **Phase 4 — November 2026 (R7 onwards)**

- Drift detection and monitoring panel
- Code emission and diff-preview push flow
- Session save/load
- First market rules deployed for live staking (paper-tracked from R4-R7, small stakes from R8-R10, scaled stakes from R11+ if validated)

## 16. Improvement backlog (deferred items)

Items considered but intentionally deferred, with the reason recorded so they are not revisited without cause:

### Deferred to Phase 5+ (post-R10, after standard framework proves out)

- **Logistic regression scoring** — potentially valuable. Threshold rules throw information away ("scored in 3 of 5" and "scored in 5 of 5" are treated identically). LR captures gradient signal that hard cutoffs miss. Risks: heavier overfitting exposure, lower interpretability, calibration requirements. Test only after standard framework has proven profitable through R10. If standard rules underperform expectations, LR moves up the priority list earlier.
- **Formal ensemble rules** — Stage 2 is already a light form of ensemble (combining Stage 1 winner with intuition filters). Formal ensembles would run multiple Stage 1 rules in parallel and pick only fixtures where all fire (voting) or weight by rule confidence (averaging). Natural extension once the library has enough vetted rules to combine.
- **Automatic feature discovery from raw data** — the app takes a curated feature list as input, not raw form matches. Auto-generating features is deferred until manual feature engineering is exhausted.

### Deferred indefinitely (out of scope by design choice)

- **Odds-aware grid search** — user is focused on data and team behavior, not price. Break-even calculations, expected-value estimates, and odds-range filters are not built into Stage 1 or Stage 2. If added later, they would live in a separate "execution" panel, not the grid searcher.
- **Multi-fixture accumulator optimization** — grid search operates on single-fixture predictions. Building accumulator legs is a downstream step outside the app's scope.
- **Team-history index for deep-form lookup** — considered as a helper to enable lookups of a team's last 12-15 matches from the cache. Not needed at current 7-match window. Revisit only if grid search consistently shows "signal is thin, more history would help."

### Kept in scope explicitly

- Sample-size floor per market (configurable)
- Baseline-relative lift as a ranking option
- Per-round breakdown of rule performance (stability check)
- H2H features (Chidi's specific concern — some teams score often but blank against each other)
- Quality-adjusted count features (avoid "scored 1 goal in each of 3 narrow-margin games" false positives)

## 17. Definitions and glossary

|                 |                                                                                         |
|-----------------|-----------------------------------------------------------------------------------------|
| <b>Baseline</b> | The market's hit rate with no filter applied. Any rule below this is worse than random. |
|-----------------|-----------------------------------------------------------------------------------------|

|                               |                                                                                                                                                                                               |
|-------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Drift</b>                  | The gap between a deployed rule's current performance and its tuned baseline. Grows over time as football conditions change.                                                                  |
| <b>Feature</b>                | A number derived from raw match data (e.g. how often a team scored in its last 5). The inputs to a rule.                                                                                      |
| <b>Flex margin</b>            | The maximum number of losses tolerated when combining multiple picks into a single betting slip. User's operating value is $\leq 4$ .                                                         |
| <b>Grid search</b>            | A method that tries every combination of threshold values for a rule and records the resulting picks, wins, and losses. Not machine learning — no model, no weights, just a systematic sweep. |
| <b>Hold-out set</b>           | Data deliberately kept out of tuning so the tuned rule can be tested honestly against fresh conditions. Also called the test set.                                                             |
| <b>Intuition rule</b>         | A hand-crafted filter with a domain-motivated rationale. Lives in the Stage 2 library. Different from Stage 1 grid-swept rules.                                                               |
| <b>is_main_league</b>         | Boolean flag on each form/H2H match indicating whether that match was played in the same competition as the upcoming fixture.                                                                 |
| <b>Orthogonality</b>          | The independence of features. Correlated features waste search compute and inflate apparent rule strength — they should be flagged.                                                           |
| <b>Out-of-sample</b>          | Performance of a tuned rule measured on data that was not used in tuning. The honest test.                                                                                                    |
| <b>Overfitting</b>            | A rule that fits random noise in the tuning data rather than real signal. Exposed by hold-out testing.                                                                                        |
| <b>Sim-ready RDS</b>          | The output of the simulator's scrape flow. A list of enriched fixtures with results attached. The primary input to the Grid Searcher.                                                         |
| <b>Stage 1</b>                | Quantitative threshold grid search finding high-volume baseline rules.                                                                                                                        |
| <b>Stage 2</b>                | Application of intuition-library filters on top of a Stage 1 winner to reduce losses to the flex margin.                                                                                      |
| <b>Tune / Validate / Test</b> | The three-way data split. Tune drives Stage 1. Validate shortlists Stage 1 winners and drives Stage 2. Test is the final honest measurement.                                                  |
| <b>Window</b>                 | How many recent matches feed a feature calculation. Typically 3, 5, or 7. Itself a tunable parameter of the grid search.                                                                      |

*This document captures the state of design as of the compile date on the cover. The plan will evolve as season data arrives and real rules get tested. Every deferred item is recoverable; every design decision has a reason recorded.*

*When development begins, hand this document to the assistant and the full context is rebuilt.*